

Designing and implementing LLM guardrails components in production environments

Mateus Devino
watsonx Platform Engineering
IBM Research
Rio de Janeiro, Brazil
mdevino@ibm.com

Evaline Ju
watsonx Platform Engineering
IBM Research
Denver, United States
evaline.ju@ibm.com

Paulo Marques Caldeira Junior
watsonx Platform Engineering
IBM Research
São Paulo, Brazil
pmej@br.ibm.com

Abstract—With advancements in generative Artificial Intelligence (AI), there has been an increasing need for tools that rely on Large Language Model (LLM)s. As these models may produce undesired answers, there is a need to prevent such events, especially in enterprise environments. Even if models are trained on safe data, user inputs and even model behavior can be unpredictable, leading to problems such as leakage of confidential data that could result in revenue loss. In this paper, we describe our experiences on developing tools for “guardrailing” LLMs. We describe how we started with a quick monolith implementation, and later transitioned to a microservices architecture. As results, we share our lessons learned throughout the process, and how the re-architecture to microservices led to runtime performance gains, easier maintenance and extensibility, and also allowed us to open source the main component of the solution, so anyone can contribute to and use it.

Index Terms—microservices, software architecture, generative AI, large language models, machine learning, guardrails, sentence tokenizers

I. INTRODUCTION

The usage of generative AI models has increased tremendously since 2022 [1], [2]. While transformers, the base architecture of generative AI models, was published in 2017 [3], it was not until the release of ChatGPT [4] that this technology was introduced to a broader audience. These technologies can be used to automate several tasks. However, the improper use of these tools may pose severe risks to society [5].

With various curious users trying out open-sourced generative models, from OpenAI’s GPT models [6] to Google’s Flan models [7], creative prompts could lead to text unsuitable for business environments, like the “grandma exploit” [8] or the “SolidGoldMagikarp” [9] glitch token.

This paper details our experience from building a monolithic guardrails architecture to breaking up the architecture into a more efficient and maintainable microservices architecture. The architectural considerations can aid in designing another LLM-based software architectures.

We use “guardrails” to refer to the additions to an LLM workflow to run detections on user prompts and generative model outputs. The use of “unsuitable” text referred to in this paper is with respect to the detector models used and their respective configured score thresholds described later, and is not meant to be a quantitative assessment of suitability in a particular context.

The rest of this paper is divided as follows: Section II describes the initial monolithic architecture and its limitations. Section III describes the microservices architecture that was created to address some of the limitations found in the monolith. Section IV details the observed consequences of the re-architecture. Section V concludes the paper, summarizing the experiences shared as well as listing future work.

II. THE FIRST APPROACH: A MONOLITHIC ARCHITECTURE

A monolith was the team’s first iteration of the guardrails architecture. It was built with existing server code, namely the REpresentational State Transfer (REST) [10] server generated with *caikit* [11] that put guardrails on LLM inputs and outputs in a production setting. The main goals were to be able to (1) detect unsuitable user prompts and issue warnings and (2) detect unsuitable generated output, regardless if the output was returned in a streaming fashion. The monolithic server called “detector models”, or models used to create detections on text. If the user prompt was not detected as unsuitable, the prompt would be sent to a separately deployed generation model server (LLM server), and the generation model output would then be analyzed by detector models in the monolith. Some detector models were trained to run only on short lengths of text, requiring that input be tokenized by sentence tokenizers. This architecture was quick to deploy to production settings and worked for its limited use case with a couple detector models. However, over time as use cases expanded, some issues were discovered.

A. Code complexity with dependencies

The monolith essentially required bundling any specific model runtime dependencies with server code. Each model used for detection usually had its own dependencies that may not overlap with other detector models. In a monolith, all of these separate dependencies had to be maintained and updated for security vulnerabilities, along with any dependencies of the server itself, such as FastAPI [12] for running a Python server.

Since many of the existing *caikit* server solutions and detector models were implemented in Python, the monolith was also written in Python. However, there were various performance concerns for concurrent usage. When one detector model’s

performance was limited, this required scaling up the entire server.

B. Adding new models

The monolith was first implemented with only an English sentence tokenizer since the first detector models used were trained on English. As use cases grew, such as when users wanted to use LLMs for translation tasks, the one English sentence tokenizer became fairly limiting. Adding support for multiple languages would at best require some refactoring and branches of code, and at worst add more dependencies and code complexity depending on tokenizer implementations. Because the detector models themselves were so embedded in the monolith, adding a detector model usually required expert knowledge on the model implementation.

C. Detections on streaming output

Many generative language models are used in a real-time nature. For a more interactive experience, users may want to see output as it is calculated, rather than potentially waiting a long time for any response.

Performing detection on streamed output proved to be one of the biggest challenges of the guardrails effort. Generally, generative model servers would stream back a few tokens at a time, whereas detector models tended to work on at least a few words, if not whole sentences or passages of text. Thus a mechanism had to be provided to aggregate text until there was enough text appropriate for each detector. This was difficult to manage if each detector model could require different amounts of text.

Additionally, any generated text had to be returned in order such that it made sense to the user, regardless of any potentially running detector models. Some detector models may process different amounts of text (e.g. sentences vs. whole documents), and other detector models may process the same amounts of text at different speeds (e.g. a simple regex sentence splitter vs. a more complicated sentence classifier). With results being returned by various detector models, the system still had to make sure the results were returned to users in the expected order. For example, one requested detector model has processed until the fourth sentence in a passage. If another second requested detector model has only processed until the second sentence in the same passage, the system would have to make sure the first detector model's results on the third and fourth sentences were not returned to the user prematurely. Instead, for the user to receive consecutive, ordered sentences, the system should make sure results were returned up to the second sentence for all detector models requested. It would certainly be very confusing if a user had requested a story and the sentences of the story came back in a different order. The monolith server then had to keep track of how much text was processed by each detector model at any point, while also keeping track of any intervening whitespace between tokens that could have been ignored or removed by tokenizers, such that all text with their intervening whitespace was correctly returned as the generative model had produced.

For ease of implementation, in the monolith, this was first implemented as text accumulation with use of a single English sentence tokenizer to identify sentences, and each detector model then classified the text of each individual sentence. This implementation was not flexible to addition of any future model that did not require either sentences or tokenization in the same language. Because of how coupled text accumulation, tokenization, intervening whitespace tracking, and detector model invocations were, maintenance and tracking down edge cases proved difficult in the monolith architecture.

III. FROM MONOLITH TO MICROSERVICES

With the limitations we found in the monolithic architecture, we decided to re-implement the guardrails architecture using microservices. The goal of this re-architecture was to better manage the current challenges, as well as prepare the system for future requirements. The microservices architecture has four types of components - generation servers, detectors, chunkers and the orchestrator. Each component type will be further described in its own section below.

A. Generation Servers

Generation servers serve LLMs that generate content. Their role is to generate some kind of output, given an user input. At the time of writing, supported generation APIs include TGIS [13], Caikit NLP [14], and OpenAI's chat completions [15]. The last chat completions API will apply for chat use cases (i.e. generating chat responses).

TGIS and Caikit NLP provide RPC interfaces [16], while OpenAI's chat completions API provides an HTTP interface. At time of writing, TGIS and Caikit NLP cannot coexist in the same configuration.

B. Detectors

Detectors in this architecture are servers that run detections on text. They generally involve serving a model like the "detector models" in the monolith architecture but do not necessarily have to. Each detector is configured with a chunker, which will be explained in the next section.

Detectors can be configured with a default score threshold, which users can override in their requests. The score threshold is used to filter out detections that may be scored low, whether in confidence or probability or any other numerical score concept. Since scores can be configured or provided per detector, reasonable defaults for each detector can be documented for use cases. This allows users to not have to necessarily wade through many detection results.

Detectors are invoked through the HTTP protocol. They can be implemented as REST APIs, with the only requirement being that they must comply with the documented detectors API [17]. Detectors are not limited to serving simple classification models. Even LLMs themselves, such as Granite Guardian [18] or Llama Guard [19] could be served as detectors.

At time of writing, there are four different types of detectors. While each detector type has a corresponding endpoint, there

is nothing preventing a single application from implementing more than one endpoint. For simplicity, we will continue to refer to each endpoint as a different detector type. Detector types are explained below.

1) **Content detectors:** Content detectors operate on text, for example a user prompt or generative model output. A use case for content detectors is analysis of Personally Identifiable Information (PII). This could be performed either on the user input (to prevent users from sending PII to a generation server), or a generative model's output (to prevent a model from leaking PII to external users).

2) **Generation analysis detectors:** Generation analysis detectors operate on both the user input and generation server output. While content detectors operate only on one or the other, generation analysis detectors analyze both at the same time. An example of a generation analysis detector is a relevance detector which compares a question asked by the user to the output provided by a model.

3) **Chat detectors:** Chat detectors operate on a chat history. A use case for chat detectors is analyzing chat history for how appropriate assistant responses are to user queries. In contrast to generation analysis detectors, chat detectors can operate on a chat message format and longer history, rather than a pair of user input and generation server output.

4) **Document context detectors:** Document context detectors operate on user input, generation server output, and also receive context documents as additional input. An example use case for this type of detector is one that performs fact checking on answers generated by an LLM, given context documents as ground truth.

C. Chunkers

Chunkers are servers that “chunk” or split content. In an LLM workflow, they can operate either on user input or generative model output. Their role is to prepare the input to be sent to detectors. As mentioned in the detectors section prior, each detector can be configured with a respective appropriate chunker. Examples of chunker models are (1) sentence tokenizers, which break text down into sentences and are appropriate for detectors that may be trained on sentences specifically or short text lengths and (2) whole document chunkers, which pass the whole input to detectors. The whole document chunkers provide information on the amount of text processed, even if the input itself is not actually “chunked”.

The term “chunkers” was chosen in place of a term like “tokenizers” since input can be divided up arbitrarily, such as code separated into functions, and does not have to be strictly limited to text. Chunkers are implemented using the RPC protocol, more specifically, the gRPC [20] implementation. This was chosen to enable the bidirectional streaming case, where streams of text characters or text tokens might be streamed from a generation server. Text can then be aggregated until an appropriate “chunk” amount (e.g. sentence) determined by the chunker, then streamed back. Users then do not have to wait for an entire model response to be available for it to be chunked and processed.

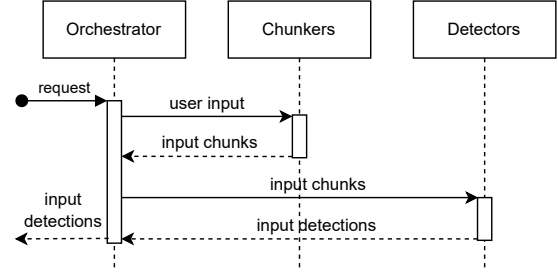


Fig. 1. Sequence diagram of the orchestrator's generation with detection endpoints when there are input detections above the threshold.

D. Orchestrator

The orchestrator component manages or “orchestrates” the communication between all other component types. It has a set of endpoints [21] that integrates chunkers, detectors, and generation servers altogether.

1) **Unary generation with detection endpoint:** The first endpoint that was implemented in the orchestrator allows for content detection on both user prompt and generative model output.

Figure 1 illustrates the processing performed for input detection. The first step is to perform chunking on user input. The orchestrator performs this by listing all chunkers associated with the “input detectors” in the request (detectors requested to run on the user prompt). This way, if two or more detectors are associated to the same chunker, only one request is sent to the chunker. If there is more than one chunker listed, chunker requests are made in parallel. If there is an error in any of the chunker calls, the orchestrator returns the error and does not proceed with the next steps. If there are no errors, the LLM workflow proceeds, and the orchestrator invokes the input detectors.

Input detectors are invoked with the appropriate chunked input. For example, if a detector is associated with a sentence tokenizer and the original input had multiple sentences, the detector input will be multiple sentences, outputted by the particular chunker. If the associated chunker is a whole document chunker, the original input will be passed to the detector. The detector will execute its logic and return its detection output to the orchestrator. If there is at least one detection above the default or user-requested threshold, the orchestrator will not proceed to the generation step and return a warning that “unsuitable input [was] detected.” If no detections above the threshold are found, the orchestrator proceeds to the generation step. Figure 2 illustrates this case.

In the generation step, the orchestrator makes a call to a generation server (often an LLM server) so that content can be generated. If an error happens during the generation call, the orchestrator does not proceed and instead propagates the error back to the user. If content was generated successfully, the orchestrator proceeds to the output chunking and detection steps.

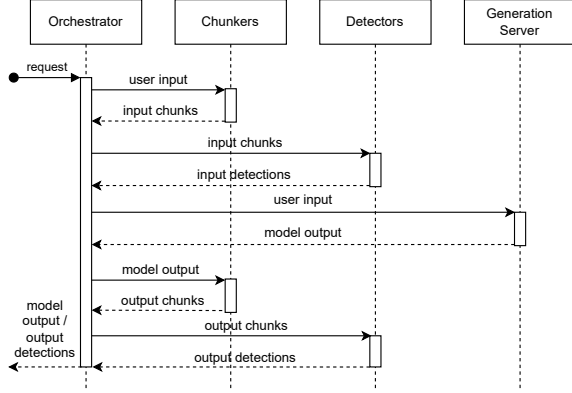


Fig. 2. Sequence diagram of the orchestrator's unary generation with detection endpoint when there are no input detections identified above the threshold. The orchestrator proceeds with calling to the generation server and invoking detections on the generated output.

The output chunking and detection steps are analogous to input chunking and detection, with the difference being that the processing is performed on the generative model output instead of the input. Once again, if there are any errors on the chunking or detection steps, the flow is interrupted, and the orchestrator propagates the error message back to the user. If there are any detections above the default or user-requested threshold in the output, the orchestrator returns the detections along with the corresponding generated text. Otherwise, only the output generated by the model is returned.

2) **Streaming generation with detection endpoint:** The steps performed in the streaming endpoint are the same as the ones in the unary endpoint, with the exception that the generation servers are requested to stream results instead of returning the whole generated content in a single response. As a consequence, the chunking requests on the generation server output make use of bidirectional streaming. This results in additional logic in the chunker implementation, as it needs to aggregate incoming stream events to form a sentence. There is also the possibility of the generation server streaming more than one sentence in a single stream event. In this case, the chunker would "chunk" or break this event into multiple sentences as required.

3) **Detection-only endpoints:** As an additional feature in comparison to the monolithic approach, we have also implemented endpoints in the orchestrator that do not provide generation. While detectors could be invoked directly with the known detectors API [17], the orchestrator also provides detection-only endpoints so that the chunking step needed for every detector can be abstracted away. These endpoints also enable orchestrator users to run multiple detectors on the same input with a single request. As there are currently four types of detectors, the orchestrator API offers four different detection-only endpoints, one for each detector type. In contrast to aforementioned endpoints, detection-only endpoints do not invoke

calls to generation servers, hence no generation is performed. This allows orchestrator users to perform their generation elsewhere and use only the orchestrator for detections.

IV. CONSEQUENCES OF THE RE-ARCHITECTURE

In the guardrails architecture, one concern was to not open-source all model server implementations, such as those with proprietary detector models. With the decoupling of components through the re-architecture, an important consequence was the orchestrator could become an open-sourced project [22]. It can now be used by anyone and receive contributions to accelerate its evolution.

The orchestrator, as a central component, is built to be performant. By definition it orchestrates results among various servers and can do so while being agnostic of server implementations. Since the APIs, including the detectors API [17], are all documented publicly, anyone can create detectors and associated chunkers for use with the orchestrator. Server implementations can be maintained separately, using frameworks or dependencies that particular developers are more comfortable working with. Users can bring their own generation servers and multiple detectors and chunkers. For example, users can select from various generation server frameworks, such as vLLM[23] or LightLLM[24].

At deployment time, all available servers can be configured for use with the orchestrator, and the end (orchestrator) user can flexibly try out various combinations of generation models and detectors. Performance optimizations for processing various server results can therefore be centralized. Measurements done from the orchestrator can easily point out bottlenecks of the other servers that the component talks to. Alternatively, a more linear chain of servers may be more difficult to validate results from server to server. Users who may want to bring their own server implementations may then have to implement their own common orchestration logic, like processing streaming content or parallelizing detector calls.

A concern we had with the re-architecture was how the network latency added by microservices would impact the overall system performance. To address this concern, we ran performance tests to validate if the network latency was meaningful. The monolith architecture had an English sentence tokenizer (chunker) and a HAP content detector. To execute the tests, we configured the monolith with eight CPUs and 16 gigabytes of RAM memory.

As the microservices architecture has multiple components, we split resources between multiple components. The orchestrator was configured with one CPU and two gigabytes of RAM memory. The English sentence tokenizer was configured with one CPU and one gigabyte of memory. The HAP detector was configured with six CPUs and 12 gigabytes of memory. All resources allocated to microservices components sum eight CPUs and 16 gigabytes of RAM memory, which are the same allocated to the monolith.

Besides the components above, each architecture used the same model in the generation server. Once we configured both architectures with comparable hardware, we executed the

TABLE I
PERFORMANCE TEST RESULTS COMPARING MONOLITH AND
MICROSERVICES ARCHITECTURES. RESPONSE TIMES ARE SHOWN IN
MILLISECONDS.

	Monolith	Microservices
Unary endpoint		
Mean	1156.12	1361.74
95th percentile	1369.05	1704.00
Streaming endpoint		
Mean	1489.40	1451.64
95th percentile	1893.45	1873.80

scenario as follows. The test started with one user making one request. As soon as the associated response returned, the user sent a new request. New users were created every 0.8 seconds, until there were 25 concurrent users. Each user performed 50 requests, summing to 1250 requests in total.

Each request contained an input of 1691 characters and requested for the HAP detection to be performed on output. Greedy search, in which the most probable token is chosen, was requested for the generation server decoding method. This ensured the generative model always returned the same message. Table I shows the results obtained from performance tests.

In Table I, rows represent response times, and columns represent the architecture. We collected the mean and 95th percentile response times, in milliseconds, for the unary and streaming endpoints. While the microservices architecture had higher response times for the unary endpoint, response times for the streaming endpoint were mildly lower.

With these tests, we verified that the network latency in the microservices architecture was not a concern. While performance did not change significantly with the re-architecture, decoupling allowed components to be scaled independently, which led to better resource utilization.

We attribute part of the latency reduction in the streaming endpoint to the use of the Rust language [25] to implement the orchestrator. In the monolith application, Python was the only language used. While Python has a huge ecosystem of machine learning libraries, it is not well-suited for highly parallel and concurrent applications because of the Global Interpreter Lock [26]. In contrast, Rust was designed for highly-parallel, memory-safe computing tasks, which makes it a great choice for a component like the orchestrator.

Overall, the microservice-based architecture has allowed separate optimizations of each server. Any chunker or detector implementations can continue to be optimized based on the selected frameworks.

V. CONCLUSION

In this experience paper, we shared our experiences implementing solutions to prevent unsuitable content created by generative models to be returned to users in production environments. We highlight the importance of “guardrailing” the generative model workflow, especially in business environments. We also shared software engineering insights acquired throughout the process of designing the guardrails framework

to be scalable, performant, and maintainable. The considerations described could benefit other software architectures including LLMs.

In production settings, the guardrails microservices architecture has been able to scale to hundreds of concurrent users from single-digit concurrent users in the initial monolithic architecture. This feat was possible due to the ability to scale resources independently for each microservice component.

Additionally, the time to deliver new detector models has decreased drastically, from the order of weeks to hours or days, since developers can flexibly choose server frameworks they are familiar with. The dependency management for components was also simplified, since each component is maintained in its own codebase.

The open-sourced orchestrator component is under active development, and users may leverage it to add “guardrails” to their LLM workflows. Some future features include support for multiple modalities and potentially support for not just “detections,” but rephrased text produced after detection or updated content for other modalities.

VI. ACKNOWLEDGMENTS

We would like to thank all team members of the Guardrails project through the various architectures - Alan Braz, Daniel Clark, Flavia Beo, Gaurav Kumbhat, Lauren McHugh, Mynhardt Burger, Nhan Hoang, Olivia Buzek, Paul Scoropan, Prashant Gupta, Rashed Bhatti, Shonda Witherspoon, Thara Palanivel, and Waleed Khan. Without the contributions of each member, this project would not be possible.

REFERENCES

- [1] S. University, “Artificial intelligence index report 2023,” 2023. [Online]. Available: <https://aiindex.stanford.edu/ai-index-report-2023/>
- [2] —, “Artificial intelligence index report 2024,” 2024. [Online]. Available: <https://aiindex.stanford.edu/report/>
- [3] A. Vaswani, “Attention is all you need,” *Advances in Neural Information Processing Systems*, 2017.
- [4] OpenAI. (2022) Introducing chatgpt. [Online]. Available: <https://openai.com/index/chatgpt/>
- [5] R. Pasupuleti, R. Vadapalli, and C. Mader, “Cyber security issues and challenges related to generative ai and chatgpt,” in *2023 Tenth International Conference on Social Networks Analysis, Management and Security (SNAMS)*, 2023, pp. 1–5.
- [6] OpenAI *et al.*, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [7] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, “Finetuned language models are zero-shot learners,” 2022. [Online]. Available: <https://arxiv.org/abs/2109.01652>
- [8] P. Verel. (2023) When ai says no, ask grandman. [Online]. Available: <https://now.fordham.edu/politics-and-society/when-ai-says-no-ask-grandma/>
- [9] S. Land and M. Bartolo, “Fishing for magikarp: Automatically detecting under-trained tokens in large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.05417>
- [10] R. T. Fielding, *Architectural styles and the design of network-based software architectures*. University of California, Irvine, 2000.
- [11] Caikit. [Online]. Available: <https://github.com/caikit/caikit>
- [12] S. Ramírez. Fastapi. [Online]. Available: <https://fastapi.tiangolo.com/>
- [13] I. Research. Tgis grpc api. [Online]. Available: <https://github.com/IBM/text-generation-inference/blob/main/proto/generation.proto>
- [14] Caikit nlp. [Online]. Available: <https://github.com/caikit/caikit-nlp>
- [15] OpenAI. Openai’s chat completion api. [Online]. Available: <https://platform.openai.com/docs/api-reference/chat>

- [16] R. Thurlow, "RPC: Remote Procedure Call Protocol Specification Version 2," RFC 5531, May 2009. [Online]. Available: <https://www.rfc-editor.org/info/rfc5531>
- [17] (2024) Orchestrator ecosystem detectors api. [Online]. Available: <https://foundation-model-stack.github.io/fms-guardrails-orchestrator/?urls.primaryName=Detector+API>
- [18] I. Padhi, M. Nagireddy, G. Cornacchia, S. Chaudhury, T. Pedapati, P. Dognin, K. Murugesan, E. Miehl, M. S. Cooper, K. Fraser, G. Zizzo, M. Z. Hameed, M. Purcell, M. Desmond, Q. Pan, I. Vejsbjerg, E. Daly, M. Hind, W. Geyer, A. Rawat, K. R. Varshney, and P. Sattigeri, "Granite guardian," 2024. [Online]. Available: <https://arxiv.org/pdf/2412.07724>
- [19] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine *et al.*, "Llama guard: Llm-based input-output safeguard for human-ai conversations," 2023. [Online]. Available: <https://arxiv.org/abs/2312.06674>
- [20] Google. (2024) grpc: A high performance, open source universal rpc framework. [Online]. Available: <https://grpc.io/>
- [21] (2024) Orchestrator ecosystem api. [Online]. Available: <https://foundation-model-stack.github.io/fms-guardrails-orchestrator/?urls.primaryName=Orchestrator+API>
- [22] (2024) Fms guardrails orchestrator. [Online]. Available: <https://github.com/foundation-model-stack/fms-guardrails-orchestrator>
- [23] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [24] (2025) Lightllm. [Online]. Available: <https://github.com/ModelTC/lightllm>
- [25] Rust programming language. [Online]. Available: <https://www.rust-lang.org/>
- [26] (2024) Global interpreter lock - python wiki. [Online]. Available: <https://wiki.python.org/moin/GlobalInterpreterLock>